HOTEL BOOKING

DATA WAREHOUSE DOCUMENTATION

Data Warehousing & Business Intelligence

TABLE OF CONTENTS

1. Dataset Selection

1.1 Introduction to the Dataset

The dataset selected for this project is a Hotel Booking Management System dataset, containing detailed records of hotel reservations, guest information, and booking outcomes. It originates from a real-world hotel environment; all personally identifiable information — names, email addresses, phone numbers, and credit card details — has been synthetically generated to ensure full privacy compliance.

To simulate a realistic, heterogeneous data environment, the dataset is distributed across multiple source file formats:

- booking_data.csv — Core booking and transactional details, including hotel type, lead time, arrival date components, length of stay, guest composition, meal preferences, market segment, distribution channel, room allocation details, deposit type, and customer classification.
- guest_data.csv — Guest-related information comprising name, email, phone number, synthetically generated credit card details, and country of origin.
- reservation_status_data.xlsx — Booking outcome and performance data such as cancellation status, Average Daily Rate (ADR), required car parking spaces, total special requests, reservation status, and the date on which that status was last updated.

1.2 Justification for Selection

The Hotel Booking dataset was selected because it satisfies all requirements for a suitable OLTP source dataset. Each record represents a single transactional booking event rather than pre-aggregated data, making it an authentic OLTP source suitable for ETL processing and dimensional modelling.

The dataset comprises over 119,000 records, providing sufficient volume to populate fact and dimension tables, construct OLAP cubes, and generate analytically meaningful reports. Its rich attribute set spans multiple business perspectives — bookings, guests, and reservation outcomes — enabling the design of a comprehensive dimensional model with dimensions for Guest, Hotel, Room, Agent, and Date.

The dataset also supports natural hierarchies, including:

- Date hierarchy: Day → Month → Year
- Geographic hierarchy: Country → Region / Continent
- Room categorisation: Room type classifications

Certain attributes, particularly within the guest domain (e.g. country and customer type), may evolve over time, making them natural candidates for Slowly Changing Dimension Type 2 implementation. Furthermore, the hotel booking domain provides a unique and realistic business

scenario, distinct from commonly used academic retail or sales datasets, thereby enhancing the practical relevance of this project.

Dataset source: https://www.kaggle.com/datasets/saadharoon27/hotel-booking-dataset

1.3 Dataset Attribute Description

The dataset contains multiple attributes representing different aspects of hotel booking transactions. For the purposes of this implementation, these attributes have been logically divided into three separate data sources to better simulate a real-world environment.

booking_data.csv

Contains the main booking-related information: hotel type, lead time (the number of days between booking and arrival), arrival date details (year, month, week number, and day), length of stay (stays_in_weekend_nights and stays_in_week_nights), guest composition (adults, children, and babies), and operational attributes such as meal type, market segment, distribution channel, reserved and assigned room types, deposit type, and customer type.

guest_data.csv

Focuses on customer-related information including the guest's name, email address, phone number, and synthetically generated credit card details. It also contains the guest's country of origin, which supports geographic analysis of the customer base.

reservation_status_data.xlsx

Contains information about the outcome of each booking: cancellation status, Average Daily Rate (ADR), required car parking spaces, total special requests, the final reservation status (e.g. Check-Out or Cancelled), and the date on which that status was last updated.

Dividing the dataset into these three sources better represents how data is stored in real enterprise systems. It also supports data integration design, facilitates the ETL process, and enables the construction of a well-structured data warehouse.

1.4 Entity-Relationship Diagram

The Entity-Relationship (ER) diagram below illustrates the logical structure of the OLTP source data following normalisation into related entities. The central entity is Booking, which references the supporting entities Guest, Hotel, Room, Date, and Agent.

Note: The ER diagram represents the logical design of the OLTP data model. In the original dataset, data was provided in a denormalised form across multiple source files. As part of data preparation, this content was logically separated and normalised into distinct entities before being loaded into the data warehouse.

NOTE	The ER diagram and star schema visualisations are available in the accompanying project repository and SSIS project files. Refer to the Visual Studio solution (HotelBooking_ETL) for full diagram exports.

Booking Entity

The Booking entity is the primary transactional entity, representing a single hotel reservation event. It holds the core measurable facts — Average Daily Rate, length of stay, number of guests, booking changes, and special requests — and is linked to Guest, Hotel, Room, and Agent through foreign key relationships. This entity forms the basis of the FactBooking table in the data warehouse.

Guest Entity

The Guest entity contains personal and demographic information about the customer who made the booking. Key attributes include name, email address, phone number, country of origin, and customer type. Guest records are stored in a separate CSV file. The country and customer type attributes are identified as slowly changing, making the Guest dimension a natural candidate for SCD Type 2 implementation.

Hotel Entity

The Hotel entity captures the type and name of the hotel where the booking was made. The dataset contains two hotel types — Resort Hotel and City Hotel — so this dimension will hold exactly two reference records.

Room Entity

The Room entity captures room-related information including the reserved room type (the code originally booked) and the assigned room type (the room provided at check-in). Enriched attributes such as room_category and room_class are introduced manually via an Excel reference file to support hierarchical analysis. The meal plan associated with the booking is also stored within this entity.

Agent Entity

The Agent entity captures booking channel details, including the agent identifier, market segment, and distribution channel. This entity is derived from booking data and represents the channel through which a reservation was made.

Date Entity

The Date entity represents the temporal aspects of each booking, derived from arrival date and reservation status date attributes that are originally stored as separate columns. These attributes are combined into a structured date format during the data preparation stage.

2. Preparation of Data Sources

2.1 Overview of Data Sources

The dataset used in this project has been organised into three separate data sources to support a realistic ETL process and simulate a heterogeneous data environment. Each source is structured to represent a different aspect of hotel operations — transactional bookings, guest information, and reservation outcomes.

2.2 Source 1 — Booking Data (booking_data.csv)

The core transactional booking information is stored in a CSV file named booking_data.csv, containing over 119,000 booking records. It includes key attributes such as hotel type, lead time, arrival date details, length of stay, guest composition, meal preferences, market segment, distribution channel, room reservation and assignment details, deposit type, and customer classification.

This dataset serves as the primary source for constructing the FactBooking table. It also contributes to the DimHotel, DimRoom, DimAgent, and DimDate dimensions during ETL processing.

2.3 Source 2 — Guest Data (guest_data.csv)

Guest-related information is stored in a CSV file named guest_data.csv. Attributes include guest name, email address, phone number, synthetically generated credit card details, and country of origin. This source is used to build the DimGuest dimension, enabling analysis of customer demographics and geographic distribution.

2.4 Source 3 — Reservation Status Data (reservation_status_data.xlsx)

Booking outcome and performance information is stored in an Excel file named reservation_status_data.xlsx. It includes cancellation status, Average Daily Rate (ADR), required car parking spaces, total special requests, reservation status, and reservation status date. This source contributes primarily to the FactBooking table, particularly for analysing booking performance, revenue trends, and cancellation behaviour.

2.5 Data Source Summary

Data Type	Source File	Format	Used For
Booking transactions	booking_data.csv	CSV	FactBooking, DimHotel, DimRoom, DimAgent, DimDate

Guest information	guest_data.csv	CSV	DimGuest
Reservation outcomes	reservation_status_data.xlsx	XLSX	FactBooking

3. Solution Architecture

3.1 Architecture Overview

The proposed Data Warehousing and Business Intelligence solution for the Hotel Booking Demand Dataset follows a layered architecture comprising four main layers: Source, Staging, Data Warehouse, and Reporting. Each layer serves a distinct purpose in the overall data flow from source systems to end-user reporting.

3.2 Source Layer

The Source Layer contains raw data extracted from multiple heterogeneous sources in different file formats:

- booking_data.csv — Transactional booking information including hotel type, arrival dates, stay duration, room details, and booking channels.
- guest_data.csv — Guest demographic and personal information such as name, country, email address, and contact details.
- reservation_status_data.xlsx — Booking outcomes including cancellation status, ADR, special requests, and reservation status details.

These files represent heterogeneous data sources that simulate real-world operational systems.

3.3 Staging Layer

The Staging Layer acts as a temporary storage area where raw data from source systems is loaded before any transformation is applied. It isolates source data from the data warehouse and provides a controlled environment for data cleaning, validation, and preparation.

Three staging tables are used in this project:

- stg_Booking — Stores raw transactional booking data extracted from booking_data.csv.
- stg_Guest — Stores guest-related information extracted from guest_data.csv.
- stg_ReservationStatus — Stores booking outcome data extracted from reservation_status_data.xlsx.

This structure ensures that each source is independently staged, allowing better data control and simplifying the ETL transformation process in SSIS.

3.4 SSIS ETL Engine

The SSIS ETL Engine is responsible for extracting data from the staging tables, applying the necessary transformations, and loading processed data into the Data Warehouse Layer.

Transformations performed at this stage include data type conversions, derived column calculations, lookup joins to resolve surrogate keys, Slowly Changing Dimension handling for the Guest dimension, and date-part assembly for the Date dimension. SSIS packages are executed in a defined sequence to ensure all dimension tables are loaded before the fact table.

3.5 Data Warehouse Layer

The Data Warehouse Layer is implemented using a Star Schema design in SQL Server. It consists of the following tables:

Dimension Tables

- DimGuest
- DimHotel
- DimRoom
- DimAgent
- DimDate

Fact Table

- FactBooking

The FactBooking table stores all measurable booking transactions, while the dimension tables store descriptive attributes used for filtering, grouping, and analysis. This structure supports efficient querying and OLAP cube processing in SSAS.

3.6 Reporting Layer

The Reporting Layer provides tools for data analysis and business intelligence reporting, enabling end users to derive meaningful insights from the data warehouse. It comprises three components:

- SQL Server Reporting Services (SSRS) — Generates structured operational and analytical reports from the data warehouse.
- SQL Server Analysis Services (SSAS) — Builds OLAP cubes supporting multidimensional analysis across different dimensions and hierarchies.
- Power BI Dashboards — Creates interactive visualisations for analysing booking trends, revenue performance, and customer behaviour.

Together, these tools transform warehouse data into actionable business insights to support effective decision-making.

4. Data Warehouse Design and Development

4.1 Dimensional Model Overview

The data warehouse for the Hotel Booking Demand Dataset is designed using a Star Schema. A central fact table — FactBooking — is surrounded by five dimension tables: DimGuest, DimHotel, DimRoom, DimAgent, and DimDate. The Star Schema was chosen over the Snowflake Schema because it delivers simpler queries, better query performance, and more straightforward implementation within SQL Server Analysis Services for OLAP cube creation.

4.2 Grain Definition

The grain of the fact table is defined as one row per hotel booking transaction. Each record in FactBooking therefore represents a single booking made by a guest at a hotel, and all measures are recorded at this level of granularity.

4.3 Fact Table — FactBooking

The FactBooking table is the central element of the dimensional model. It holds foreign keys referencing all five dimension tables, along with the following measures:

- lead_time — Number of days between booking and arrival.
- stays_weekend_nights and stays_week_nights — Length of stay components.
- total_stays — Derived measure: sum of weekend and weekday nights.
- total_guests — Derived measure: sum of adults, children, and babies.
- adr — Average Daily Rate (revenue per occupied room per night).
- total_revenue — Derived measure: ADR multiplied by total stays.
- booking_changes — Number of amendments made to the booking.
- days_in_waiting_list — Days the booking spent on a waiting list.
- total_special_requests — Number of special requests made by the guest.
- is_canceled — Boolean flag indicating whether the booking was cancelled.
- reservation_status — Final state of the booking (e.g. Check-Out, Cancelled).

Three additional columns support the accumulating fact table pattern introduced in Task 6: accm_txn_create_time, accm_txn_complete_time, and txn_process_time_hours.

4.4 Dimension Tables

DimGuest

Stores personal and demographic information about the guest who made the booking. This dimension is implemented as a Slowly Changing Dimension Type 2: when key attributes such as country or customer type change over time, a new record is inserted rather than overwriting the existing one. The effective_date, expiry_date, and is_current columns track the full history of changes. The guest_id column serves as the natural key; guest_sk is the surrogate key.

DimHotel

A simple reference dimension storing hotel name and hotel type. The dataset contains only two hotel types — Resort Hotel and City Hotel — so this dimension holds exactly two records.

DimRoom

Stores room-related attributes including the reserved room type and the assigned room type. Enriched attributes such as room_category and room_class support hierarchical analysis. The room hierarchy flows from reserved room type up to room category and then to room class.

DimAgent

Stores booking channel information including agent identifier, market segment, distribution channel, and deposit type. This dimension supports analysis of booking patterns across different market segments and distribution channels.

DimDate

Stores date-related attributes derived from the arrival date columns in the source data. It supports hierarchies from day level up to year level, including day of month, month number, month name, quarter, year, and week number. A Boolean flag named is_weekend identifies whether the date falls on a weekend.

4.5 Slowly Changing Dimension

DimGuest is implemented as a Slowly Changing Dimension Type 2. When the country or customer type of a guest changes, the existing record is expired by setting expiry_date to the date of the change and the is_current flag to 0. A new record is then inserted with the updated values, a new effective_date, and is_current set to 1. This approach preserves the full history of guest attribute changes, enabling accurate historical analysis in the data warehouse.

4.6 Design Assumptions

The following assumptions were made during the design of the dimensional model:

- The arrival date is constructed during ETL by combining arrival_date_year, arrival_date_month, and arrival_date_day_of_month from the source data.
- total_stays is derived by adding stays_weekend_nights and stays_week_nights.

- total_revenue is derived by multiplying adr by total_stays.
- Room hierarchy attributes (room_category and room_class) are not present in the original dataset and are manually assigned based on room type code.
- Null values in the agent_id column indicate direct bookings made without a booking agent.
- All surrogate keys are generated automatically using the SQL Server IDENTITY property.

5. ETL Development

5.1 ETL Process Overview

The ETL process is implemented using Microsoft SQL Server Integration Services (SSIS) and follows a layered approach. A central master package (Package.dtsx) orchestrates all ETL workflows in a controlled sequence:

1. Staging Layer — raw data ingestion from source files into staging tables.
2. Dimension Load Layer — transformation and loading of dimension tables.
3. Fact Load Layer — surrogate key resolution and FactBooking population.
4. Master Control Package — orchestration of all layers in dependency order.

5.2 Data Profiling

A Data Profiling Task was executed in SSIS prior to ETL processing to analyse the quality, structure, and consistency of the source data. The profiling examined:

- Null value distribution
- Data type consistency
- Value frequency distribution
- Uniqueness of key columns
- Data anomalies and inconsistencies

The profiling results confirmed that booking and guest data were largely complete with no major structural inconsistencies. Month values required standardisation from string to numeric format, and the data was determined to be suitable for ETL processing after the staging load was complete.

5.3 Staging Layer (staging.dtsx)

The purpose of the Staging Layer package is to extract raw data from multiple source files and load it into staging tables without applying transformations. The process follows two steps:

5. Truncate all staging tables using an Execute SQL Task to ensure a clean load.
6. Load data into staging tables using Data Flow Tasks.

Within each Data Flow Task, data flows from a source connector (Flat File Source or Excel Source) directly to an OLE DB Destination corresponding to the appropriate staging table: stg_Booking, stg_Guest, or stg_ReservationStatus. All three source files successfully loaded 119,390 rows into their respective staging tables, as confirmed by validation queries in SQL Server Management Studio.

5.4 Dimension Load Layer (Dimensions.dtsx)

The Dimension Load Layer package loads all five dimension tables from the staging data. Dimensions are loaded in the following order to satisfy foreign key dependencies:

7. DimDate
8. DimHotel
9. DimRoom
10. DimAgent
11. DimGuest (SCD Type 2)

5.4.1 DimHotel

Room-related attributes are extracted from the staging booking table and loaded into DimHotel. A Derived Column transformation is applied to clean hotel attribute values using the TRIM function, removing any leading or trailing whitespace to ensure data consistency before loading.

5.4.2 DimRoom

Room-related attributes are extracted from the staging booking data and loaded into DimRoom. A Derived Column transformation standardises the reserved_room_type and assigned_room_type values by converting them to uppercase, ensuring consistency across analytical queries.

5.4.3 DimAgent

Booking channel information is extracted and loaded into DimAgent. A Sort transformation organises records by market segment and distribution channel in ascending order. A subsequent Derived Column transformation generates a composite attribute named agent_segment_key by concatenating market_segment, a hyphen delimiter, and distribution_channel. This composite key supports analytical grouping of bookings by channel.

5.4.4 DimGuest — SCD Type 2 Implementation

The DimGuest table implements Slowly Changing Dimension Type 2 to preserve a complete historical record of guest attribute changes. The data flow follows five distinct stages:

1. Data Extraction

Data is extracted from the staging table stg_Guest using an OLE DB Source component.

2. Lookup Transformation

A Lookup transformation checks whether each guest record already exists in DimGuest by matching on the natural key and the core attributes: guest_id, name, email, phone_number, credit_card, and country. This step distinguishes new records from existing ones.

3. Conditional Split — Change Detection

A Conditional Split transformation detects whether any tracked attributes have changed since the last load. Records are routed to the "changed" output when any of the following differ between source and warehouse: name, email, phone_number, credit_card, or country.

4. Expire Old Record

For changed records, the existing warehouse record is marked as expired. A Derived Column transformation sets Expire_is_current = 0 and Expire_effective_end = GETDATE(), and an OLE DB Command applies this update to the relevant row in DimGuest.

5. Insert New Version

After expiring the old record, a new version of the guest record is inserted. A Derived Column transformation sets is_current = 1, effective_start = GETDATE(), and effective_end = '9999-12-31'. The final data is written to DimGuest via an OLE DB Destination, ensuring the latest record remains active while full historical data is preserved.

SCD TYPE 2	The implementation ensures historical tracking of guest changes, preservation of all record versions, accurate time-based analysis, and correct use of SSIS transformations including Lookup, Conditional Split, Derived Column, and OLE DB Command.

5.4.5 DimDate

The DimDate table provides a structured date hierarchy supporting efficient analysis of booking trends over time, including monthly patterns, seasonal trends, and year-over-year comparisons.

Data Extraction

Data is extracted from stg_Booking using an OLE DB Source with a DISTINCT query, removing duplicates to ensure the uniqueness of each date record.

Data Standardisation

Month names are converted to numeric values using a CASE expression within the OLE DB Source SQL command (January = 1, February = 2, through December = 12), ensuring consistency in date representation across all records.

Surrogate Key Generation

A Derived Column transformation generates the surrogate date key using the expression (date_year * 10000) + (month_number * 100) + day_of_month, producing a unique integer in YYYYMMDD format, which serves as the primary key of the DimDate table.

Data Load

The transformed records are loaded into DimDate via an OLE DB Destination component. Validation queries in SSMS confirmed successful population of all date hierarchy attributes.

5.5 Fact Load Layer (Fact.dtsx)

The FactBooking table integrates data from multiple sources, resolves surrogate keys from all dimension tables, and loads the final analytical records into the data warehouse.

Source Integration

Two OLE DB sources are used: stg_Booking, which provides core booking transactional data, and stg_ReservationStatus, which contains reservation outcome information. Both datasets are sorted before joining — stg_Booking by stg_booking_id (ascending) and stg_ReservationStatus by stg_reservation_id (ascending) — to enable proper alignment for merge operations.

A Merge Join transformation then combines both sorted datasets into a unified transactional dataset using an inner join on matching booking and reservation identifiers, integrating booking details with their corresponding reservation status information.

Surrogate Key Resolution

After merging, the data passes through five Lookup transformations, each replacing a natural key with the corresponding surrogate key from a dimension table:

- DimHotel → hotel_key
- DimGuest → guest_key
- DimRoom → room_key
- DimAgent → agent_key
- DimDate → date_key

This lookup chain maintains referential integrity throughout the data warehouse schema.

Accumulating Fact Column

A Derived Column transformation captures the transaction creation timestamp using GETDATE(), populating the accm_txn_create_time column. This records the exact moment each fact record is inserted into the warehouse, forming the basis for processing time calculations in Task 6.

Error Handling

An OnError event handler is implemented at the package level within Fact.dtsx. Any runtime error during ETL execution is captured and logged into an ETL_ErrorLog table, along with the error message, timestamp, and package name. This provides baseline monitoring and debugging support throughout the ETL process.

6. ETL Development — Accumulating Fact Tables

6.1 Overview

The accumulating fact table pattern was implemented to track the full lifecycle of booking transactions from creation through to completion. This approach enables analysis of processing duration and transaction status over time, providing insight into operational efficiency.

To support this pattern, the FactBooking table was extended with three additional columns:

- accm_txn_create_time — Stores the timestamp when the transaction record is initially created in the data warehouse.
- accm_txn_complete_time — Stores the timestamp when the transaction is completed (populated by a subsequent update process).
- txn_process_time_hours — Stores the total processing duration in hours between creation and completion.

6.2 Step 1 — Extending the Fact Table Schema

The FactBooking table schema was extended using an ALTER TABLE statement to add the three accumulating fact columns:

- accm_txn_create_time — defined as DATETIME, NOT NULL.
- accm_txn_complete_time — defined as DATETIME, initially NULL (unknown at load time).
- txn_process_time_hours — defined as a computed column using DATEDIFF(HOUR, accm_txn_create_time, accm_txn_complete_time), calculated automatically by SQL Server.

These columns enable incremental updates to transaction records as new information becomes available, without requiring a full reload of the fact table.

6.3 Step 2 — Setting accm_txn_create_time During Initial Load

During the initial fact table load in Fact.dtsx, the accm_txn_create_time column is populated using a Derived Column transformation that applies GETDATE(). This ensures that every record inserted into FactBooking contains an accurate timestamp indicating when it was first loaded into the data warehouse.

6.4 Step 3 — Preparing the Completion Dataset

A separate dataset named txn_complete.csv was created to simulate the late arrival of transaction completion data. This file represents a realistic scenario in which booking completion events occur

after the initial data load and are received as a subsequent update feed. The file contains two columns: booking_key (the natural key identifying the transaction) and accm_txn_complete_time (the timestamp of completion).

6.5 Step 4 — Update_AccumulatingFact.dtsx Package

A dedicated SSIS package named Update_AccumulatingFact.dtsx was developed to process the completion dataset and update the FactBooking table accordingly. The package data flow operates as follows:

- A Flat File Source reads booking_key and accm_txn_complete_time from the txn_complete.csv file.
- A Derived Column transformation (included for demonstration purposes) generates an accm_txn_create_time value using GETDATE(). Note that this generated value is not used in the update logic; the existing value already stored in the database is used instead.
- An OLE DB Command performs row-by-row updates on FactBooking using the following SQL update statement:

SQL UPDATE	UPDATE FactBooking SET accm_txn_complete_time = ?, txn_process_time_hours = DATEDIFF(HOUR, accm_txn_create_time, ?) WHERE booking_key = ? AND accm_txn_complete_time IS NULL

The condition accm_txn_complete_time IS NULL ensures that records which have already been completed are not overwritten on subsequent package executions, preventing duplicate updates.

6.6 Step 5 — Processing Time Calculation

The transaction processing duration is calculated dynamically during the update phase using the SQL DATEDIFF function: DATEDIFF(HOUR, accm_txn_create_time, accm_txn_complete_time). This produces the number of elapsed hours between the initial warehouse load and the recorded completion event, ensuring accurate measurement of the end-to-end transaction lifecycle for each booking.

6.7 Validation

Following execution of the update package, validation queries were run in SQL Server Management Studio to confirm correctness. Validation checks included:

- Verifying that accm_txn_complete_time was updated correctly for the expected booking keys.
- Confirming that txn_process_time_hours was calculated and populated accurately.
- Ensuring no duplicate or previously completed records were overwritten.

All checks passed successfully, confirming that the accumulating fact table implementation behaves as intended.

6.8 Summary

The accumulating fact table implementation delivers the following capabilities:

- Tracks the full transaction lifecycle from initial creation to confirmed completion.
- Handles late-arriving data through a dedicated, independently executable ETL package.
- Calculates processing duration dynamically at the time of update.
- Prevents overwriting of already-completed records through a conditional update constraint.
- Demonstrates advanced ETL techniques using SSIS components including Flat File Source, Derived Column, and OLE DB Command.

End of Document